

Postgresql- procedimientos almacenados y funciones.

vamos a crear la aplicacion completa con postgres 18 crando un esquema que se llame ventas. un rol usuario. y un rol con login para hacer pedidos. Una tabla clientes con un campo que se llame region.(US,EU). los usuarios solo podran listar y insertar registros cuando la region sea igual a la de una variable llamada app_region. por la tanto hay que hacer rls y una politica de seguridad. tambien hay que hacer un triguier para que actualize el stock. y si ves algo que que se me pasa lo agregas tambien. todo paso por paso y explicado.

Diseñaremos la arquitectura completa en PostgreSQL 18 cubriendo todos los aspectos requeridos: creación del esquema, roles con distintas jerarquías de permisos (administrador vs. usuario de aplicación), Row-Level Security (RLS) basado en variables de sesión (`app_region`), tablas relacionadas, trigger de validación/descuento de stock y funciones helper.

Arquitectura General del Ejemplo

1. Esquema: `ventas`
2. Roles:
Medium
 - `rol_usuario_base`: Rol sin login para agrupación de permisos.
 - `app_ventas_user`: Rol con LOGIN que usará la aplicación para conectarse.
3. Tablas:
Medium
 - `ventas.clientes`: Con columna `region` (US o EU). Posee políticas RLS para restringir SELECT e INSERT.
 - `ventas.productos`: Catálogo de inventario con control de stock.
 - `ventas.pedidos`: Histórico de compras con FK hacia `clientes` y `productos`.
4. Seguridad / RLS: Filtro activo contra la variable de sesión `app.app_region`.
Bytebase
5. Automatización: Trigger que liquida stock en `productos` previa verificación de disponibilidad.

Paso 1: Esquema y Configuración de Roles

Creamos el esquema `ventas`, el rol genérico para permisos y el usuario de la aplicación.

SQL

```
-- 1. Crear el esquema
CREATE SCHEMA IF NOT EXISTS ventas;
```

```
-- 2. Crear un rol de grupo (sin login) para administrar permisos de manera
limpia
CREATE ROLE rol_usuario_base NOLOGIN;

-- 3. Crear el rol con LOGIN que utilizará el backend de la aplicación
CREATE ROLE app_ventas_user WITH LOGIN PASSWORD 'PasswordSegura123!';

-- Asignar el rol base al usuario de login
GRANT rol_usuario_base TO app_ventas_user;

-- Otorgar permisos de uso sobre el esquema
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
```

Paso 2: Creación de las Tablas y Constraints

Creamos las tres tablas en el esquema `ventas`. Añadimos restricciones `CHECK` para validar la región y que el stock/precios sean válidos.

SQL

```
-- Tabla de Clientes
CREATE TABLE ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de Productos
CREATE TABLE ventas.productos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL CHECK (precio >= 0),
    stock INT NOT NULL CHECK (stock >= 0)
);

-- Tabla de Pedidos
CREATE TABLE ventas.pedidos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id INT NOT NULL REFERENCES ventas.clientes(id) ON DELETE RESTRICT,
    producto_id INT NOT NULL REFERENCES ventas.productos(id) ON DELETE RESTRICT,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Paso 3: Asignación de Permisos de Tablas y Secuencias

Asignamos permisos sobre las tablas al `rol_usuario_base`.

SQL

```
-- Permisos de lectura/escritura en las tablas
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA ventas TO rol_usuario_base;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA ventas TO rol_usuario_base;
```

Paso 4: Implementación de Row-Level Security (RLS)

Configuramos RLS en `ventas.clientes`. La consulta obtendrá dinámicamente la variable personalizada `app.app_region` mediante la función de PostgreSQL `current_setting('app.app_region', true)`.

SQL

```
-- 1. Habilitar RLS en la tabla clientes
ALTER TABLE ventas.clientes ENABLE ROW LEVEL SECURITY;

-- 2. Política para SELECT: Solo listar clientes cuya región coincida con
app.app_region
CREATE POLICY policy_clientes_select ON ventas.clientes
    FOR SELECT
    TO rol_usuario_base
    USING (region = current_setting('app.app_region', true));

-- 3. Política para INSERT: Solo permitir insertar clientes cuya región sea
app.app_region
CREATE POLICY policy_clientes_insert ON ventas.clientes
    FOR INSERT
    TO rol_usuario_base
    WITH CHECK (region = current_setting('app.app_region', true));
Nota importante:
```

- USING aplica para el filtrado de filas existentes (SELECT).
- WITH CHECK aplica a las filas que entran a la base de datos (INSERT / UPDATE).
- El segundo parámetro true en current_setting(..., true) evita que lance un error crítico si la variable aún no fue declarada en la sesión (retornará NULL).

Paso 5: Trigger para Actualización Automática de Stock

Creamos la función en PL/pgSQL y la asociamos como BEFORE INSERT en ventas.pedidos.

1. Función con SECURITY DEFINER y Bloqueo Selectivo

Usamos SECURITY DEFINER para asegurar que el usuario ejecutor pueda modificar el stock aunque RLS o permisos directos estén limitados, y aplicamos FOR UPDATE para evitar problemas de concurrencia (Race Conditions).

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()
RETURNS TRIGGER
SECURITY DEFINER
AS $$
DECLARE
    v_stock_actual INT;
BEGIN
    -- Bloquear y obtener el stock actual del producto para evitar race
conditions en peticiones simultáneas
    SELECT stock INTO v_stock_actual
    FROM ventas.productos
    WHERE id = NEW.producto_id
    FOR UPDATE;

    -- Validar existencia del producto
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El producto con ID % no existe.', NEW.producto_id;
    END IF;

    -- Validar disponibilidad
    IF v_stock_actual < NEW.cantidad THEN
```

```

        RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:
%, Solicitado: %',
        NEW.producto_id, v_stock_actual, NEW.cantidad;
    END IF;

    -- Actualizar stock
    UPDATE ventas.productos
    SET stock = stock - NEW.cantidad
    WHERE id = NEW.producto_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

2. Declaración del Trigger

SQL

```

CREATE TRIGGER tr_actualizar_stock_pedido
BEFORE INSERT ON ventas.pedidos
FOR EACH ROW
EXECUTE FUNCTION ventas.fn_descontar_stock_pedido();

```

Paso 6: Demostración Completa de Uso y Verificación

A continuación, simula el comportamiento de la aplicación conectándose con el rol `app_ventas_user` y ajustando la variable de región.

1. Cargar datos iniciales (Ejecutar como Admin / Superusuario)

SQL

```

-- Insertar productos en inventario
INSERT INTO ventas.productos (nombre, precio, stock) VALUES
('Laptop Pro 16', 1500.00, 10),
('Teclado Mecánico', 120.00, 5);

```

2. Iniciar sesión simulada como `app_ventas_user`

SQL

```

-- Cambiamos de rol dentro de la sesión de PostgreSQL
SET ROLE app_ventas_user;

-- Intentamos listar clientes sin definir la región (Resultado: 0 filas por RLS)
SELECT * FROM ventas.clientes;

-----
-- CONFIGURACIÓN DE REGIÓN DE APLICACIÓN: US
-----
SET LOCAL app.app_region = 'US';

-- Insertar un cliente en la región US (Permitido)
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');

-- Intentar insertar un cliente de la región EU estando en región US (Bloqueado
por RLS)
-- ERROR: new row violates row-level security policy for table "clientes"
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');

-- Consultar clientes (Solo devolverá registros de 'US')
SELECT * FROM ventas.clientes;

```

```

-----
-- REALIZAR UN PEDIDO Y VERIFICAR STOCK
-----
-- Insertar un pedido de 3 Laptops (Producto ID 1) para John Doe (Cliente ID 1)
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 1, 3);

-- Consultar el stock actualizado de la Laptop (Bajó de 10 a 7)
SELECT * FROM ventas.productos WHERE id = 1;

-- Intentar hacer un pedido que excede el stock (Producto ID 2 tiene 5 unidades)
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 2, 10);
-- ERROR: Stock insuficiente para el producto ID 2. Disponible: 5, Solicitado:
10

-----
-- CAMBIO DE REGIÓN EN LA APLICACIÓN: EU
-----
SET LOCAL app.app_region = 'EU';

-- Insertar cliente de la región EU
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');

-- Al consultar, solo veremos a Pierre Dubois (John Doe queda filtrado)
SELECT * FROM ventas.clientes;

```

Detalles Adicionales Incluidos

1. **SET LOCAL app.app_region:** Usar SET LOCAL asegura que la variable de región solo persista durante la transacción actual, lo cual previene fugas de contexto cuando se utilizan pools de conexiones (como PgBouncer o HikariCP).
2. **SECURITY DEFINER** en la función del Trigger: Permite descontar el inventario en la tabla productos de forma segura aunque el rol ejecutor no tenga privilegios administrativos directos.
3. **Control de Concurrencia (FOR UPDATE):** Previene race conditions (condiciones de carrera) si dos usuarios intentan pedir el último ítem en inventario exactamente al mismo tiempo.

Bytebase

¿Quieres ver cómo integrar esta configuración con un pool de conexiones en Node.js, Python o Java?

Sí

otra política para que no se puedan hacer pedidos si no coincide la región

Para asegurarte de que un cliente solo pueda realizar pedidos en su propia región (es decir, un usuario conectado con `app.app_region = 'US'` no pueda registrar un pedido usando el `cliente_id` de un cliente de la región 'EU'), tenemos dos formas principales de implementarlo en PostgreSQL:

1. Mediante Row-Level Security (RLS) en la tabla **pedidos** (A nivel de políticas de seguridad SQL).
2. Dentro del Trigger de la tabla **pedidos** (A nivel de lógica de negocio en la base de datos).

A continuación te muestro ambas opciones explicadas paso a paso.

Opción 1: Mediante Política RLS en ventas.pedidos (Recomendada)

Podemos activar RLS en la tabla `ventas.pedidos` y crear una política que valide, mediante un `EXISTS`, que el `cliente_id` del pedido pertenece a la región configurada en `app.app_region`.

1. Habilitar RLS en ventas.pedidos

SQL

```
ALTER TABLE ventas.pedidos ENABLE ROW LEVEL SECURITY;
```

2. Crear la Política de INSERT

SQL

```
CREATE POLICY policy_pedidos_insert ON ventas.pedidos
FOR INSERT
TO rol_usuario_base
WITH CHECK (
    EXISTS (
        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
              AND c.region = current_setting('app.app_region', true)
    )
);
```

3. Crear la Política de SELECT (Opcional pero recomendada)

Para que los usuarios tampoco puedan listar los pedidos de clientes de otras regiones:

SQL

```
CREATE POLICY policy_pedidos_select ON ventas.pedidos
FOR SELECT
TO rol_usuario_base
USING (
    EXISTS (
        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
              AND c.region = current_setting('app.app_region', true)
    )
);
```

Opción 2: Validar la Región dentro del Trigger de Pedidos

Si prefieres centralizar la lógica de negocio y dar un mensaje de error más explícito cuando la región no coincida, podemos modificar la función que ya teníamos en el trigger `fn_descontar_stock_pedido`:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()
RETURNS TRIGGER
SECURITY DEFINER
AS $$
DECLARE
    v_stock_actual INT;
    v_region_cliente VARCHAR(2);
    v_app_region VARCHAR(2);
BEGIN
```

```

-- 1. Obtener la región configurada en la sesión actual
v_app_region := current_setting('app.app_region', true);

-- 2. Obtener la región del cliente asociado al pedido
SELECT region INTO v_region_cliente
FROM ventas.clientes
WHERE id = NEW.cliente_id;

-- Validar si el cliente existe
IF NOT FOUND THEN
    RAISE EXCEPTION 'El cliente con ID % no existe.', NEW.cliente_id;
END IF;

-- 3. VALIDACIÓN DE REGIÓN: Si la región del cliente no coincide con
app.app_region, rechazar el pedido
IF v_region_cliente IS DISTINCT FROM v_app_region THEN
    RAISE EXCEPTION 'Operación no permitida: El cliente ID % pertenece a la
región %, pero la sesión actual está configurada en %.',
NEW.cliente_id, v_region_cliente, COALESCE(v_app_region, 'SIN
CONFIGURAR');
END IF;

-- 4. Bloquear y verificar stock del producto
SELECT stock INTO v_stock_actual
FROM ventas.productos
WHERE id = NEW.producto_id
FOR UPDATE;

IF NOT FOUND THEN
    RAISE EXCEPTION 'El producto con ID % no existe.', NEW.producto_id;
END IF;

IF v_stock_actual < NEW.cantidad THEN
    RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:
%, Solicitado: %',
NEW.producto_id, v_stock_actual, NEW.cantidad;
END IF;

-- 5. Descontar stock
UPDATE ventas.productos
SET stock = stock - NEW.cantidad
WHERE id = NEW.producto_id;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Prueba de Funcionamiento

Simulemos el escenario donde intentamos crear un pedido para un cliente de la región 'EU' cuando la app está configurada en la región 'US':

SQL

```

-- Cambiamos al usuario de la aplicación
SET ROLE app_ventas_user;

-- Caso de uso:
-- Supongamos que en la BD existe:
-- Cliente ID 1 (John Doe) -> Región 'US'
-- Cliente ID 2 (Pierre Dubois) -> Región 'EU'

```

-- CONFIGURACIÓN DE REGIÓN EN LA SESIÓN: US

```

-----
SET LOCAL app.app_region = 'US';

-- 1. Intentar hacer un pedido para John Doe (Cliente US - ID 1) -> PERMITIDO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 1, 1);

-- 2. Intentar hacer un pedido para Pierre Dubois (Cliente EU - ID 2) estando en
sesión US -> BLOQUEADO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (2, 1, 1);

-- Si usas Opción 1 (RLS), responderá:
-- ERROR: new row violates row-level security policy for table "pedidos"

-- Si usas Opción 2 (Trigger), responderá:
-- ERROR: Operación no permitida: El cliente ID 2 pertenece a la región EU, pero
la sesión actual está configurada en US.

```

¿Cuál opción elegir?

- Opción 1 (RLS): Es la mejor práctica estándar en PostgreSQL si quieres mantener las políticas de seguridad nativas e invisibles para la lógica de la aplicación.
- Opción 2 (Trigger): Es preferible si deseas controlar los mensajes de error personalizados para que el backend reconozca exactamente por qué falló la inserción.

¿Quieres que añadamos políticas RLS para UPDATE y DELETE en las tablas de pedidos y clientes?

Sí

ahora creame un listado de consultas para poder ver todo por sql. roles, esquemas, politicas, trigger, tablas por esquema, permisos etc.

Aquí tienes un listado completo de consultas SQL nativas (utilizando el catálogo del sistema de PostgreSQL `pg_catalog` e `information_schema`) para auditar e inspeccionar roles, esquemas, tablas, políticas de RLS, triggers y permisos.

1. Consultas de Roles y Usuarios

SQL

```

-- Ver todos los roles, si tienen LOGIN, si son superusuarios y sus miembros
SELECT
    r.rolname AS nombre_rol,
    r.rolcanlogin AS permite_login,
    r.rolsuper AS es_superuser,
    ARRAY(
        SELECT m.rolname
        FROM pg_roles m
        JOIN pg_auth_members am ON m.oid = am.roleid
        WHERE am.member = r.oid
    ) AS pertenece_a_grupos
FROM pg_roles r
ORDER BY r.rolname;

```

2. Consultas de Esquemas

SQL


```
-- Listar todos los esquemas creados (excluyendo esquemas del sistema)
SELECT
    schema_name AS esquema,
    schema_owner AS propietario
FROM information_schema.schemata
WHERE schema_name NOT IN ('pg_catalog', 'information_schema')
    AND schema_name NOT LIKE 'pg_temp%'
ORDER BY schema_name;
```

3. Consultas de Tablas por Esquema

SQL

```
-- Listar todas las tablas indicando esquema, nombre y si tienen RLS habilitado
SELECT
    schemaname AS esquema,
    tablename AS tabla,
    tableowner AS propietario,
    rowsecurity AS rls_habilitado,
    forcerowsecurity AS rls_forzado
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY schemaname, tablename;
```

4. Consultas de Políticas de Seguridad (RLS)

SQL

```
-- Ver todas las políticas de Row-Level Security definidas en las tablas
SELECT
    schemaname AS esquema,
    tablename AS tabla,
    policyname AS nombre_politica,
    roles AS roles_afectados,
    cmd AS operacion, -- SELECT, INSERT, UPDATE, DELETE o ALL
    qual AS condicion_using,
    with_check AS condicion_with_check
FROM pg_policies
WHERE schemaname = 'ventas' -- Puedes quitar el WHERE para ver todos los
esquemas
ORDER BY schemaname, tablename, policyname;
```

5. Consultas de Triggers y Funciones

SQL

```
-- Listar todos los triggers configurados, su tabla y la función que ejecutan
SELECT
    event_object_schema AS esquema,
    event_object_table AS tabla,
    trigger_name AS nombre_trigger,
    action_timing AS momento, -- BEFORE, AFTER, INSTEAD OF
    event_manipulation AS evento, -- INSERT, UPDATE, DELETE
    action_statement AS funcion_ejecutada
FROM information_schema.triggers
WHERE event_object_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY event_object_schema, event_object_table;

-- Ver el código fuente de las funciones personalizadas (como la del trigger)
SELECT
    n.nspname AS esquema,
    p.proname AS nombre_funcion,
    pg_get_functiondef(p.oid) AS codigo_fuente
FROM pg_proc p
JOIN pg_namespace n ON n.oid = p.pronamespace
```

```
WHERE n.nspname = 'ventas';
```

6. Consultas de Permisos (ACLs) sobre Tablas

SQL

```
-- Ver los permisos explícitos otorgados a roles sobre las tablas de un esquema
SELECT
    grantee AS rol_otorgado,
    table_schema AS esquema,
    table_name AS tabla,
    string_agg(privilege_type, ', ') AS permisos
FROM information_schema.role_table_grants
WHERE table_schema = 'ventas'
GROUP BY grantee, table_schema, table_name
ORDER BY table_schema, table_name, grantee;
```

7. Consultas de Claves Foráneas y Relaciones

SQL

```
-- Ver las relaciones entre tablas (Foreign Keys)
SELECT
    tc.table_schema AS esquema,
    tc.table_name AS tabla_origen,
    kcu.column_name AS columna_origen,
    ccu.table_name AS tabla_destino,
    ccu.column_name AS columna_destino,
    tc.constraint_name AS nombre_restriccion
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
    AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
    AND ccu.table_schema = tc.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
    AND tc.table_schema = 'ventas';
```

Bonus: Atajos en la consola interactiva `psql`

Si estás utilizando la terminal con el comando `psql`, estos comandos aceleran la inspección sin necesidad de escribir consultas complejas:

- `\du` : Listar roles y usuarios.
- `\dn` : Listar esquemas.
- `\dt ventas.*` : Listar tablas del esquema `ventas`.
- `\d+ ventas.clientes` : Ver la estructura completa de la tabla `clientes` (columnas, FK, triggers y políticas RLS).
- `\df+ ventas.*` : Listar funciones del esquema `ventas` y ver su código.

¿Quieres una consulta para auditar las conexiones activas y las variables de sesión actuales?

Sí

ERROR: no existe la relación «ventas.clientes»

Este error ocurre principalmente por dos razones comunes:

1. Olvidaste ejecutar el comando `CREATE SCHEMA ventas;` o la tabla `clientes` dentro de ese esquema antes de hacer la consulta.
2. Estás ejecutando la consulta como un usuario/rol que no tiene permisos de uso (`USAGE`) sobre el esquema `ventas`, por lo que PostgreSQL actúa como si la tabla no existiera para proteger la seguridad.

Solución Paso a Paso

1. Verifica si el esquema y la tabla realmente existen

Ejecuta esta consulta con un usuario administrador/superusuario (por ejemplo, `postgres`):

SQL

```
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_name = 'clientes';
```

- Si la consulta no devuelve nada: La tabla no ha sido creada. Ejecuta primero la creación del esquema y las tablas:

SQL

```
CREATE SCHEMA IF NOT EXISTS ventas;
```

```
CREATE TABLE ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. Otorga los permisos necesarios al usuario/rol

Si la tabla sí existe pero te da ese error al ejecutar desde el rol de la aplicación (`app_ventas_user` o `rol_usuario_base`), debes otorgarle permisos explícitos sobre el esquema y la tabla desde tu usuario administrador:

SQL

```
-- 1. Dar permiso para "ver/usar" el esquema ventas
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;

-- 2. Dar permisos sobre las tablas dentro del esquema
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO
rol_usuario_base;

-- 3. Asegurar que app_ventas_user heredó el rol base
GRANT rol_usuario_base TO app_ventas_user;
```

3. Revisa tu `search_path` (Opcional)

Si prefieres escribir `SELECT * FROM clientes;` en lugar de `SELECT * FROM ventas.clientes;`, puedes agregar el esquema `ventas` a la ruta de búsqueda de la sesión:

SQL

```
SET search_path TO ventas, public;
```

igual: -- 1. Dar permiso para "ver/usar" el esquema ventas

```
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
```

-- 2. Dar permisos sobre las tablas dentro del esquema

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO rol_usuario_base;
```

-- 3. Asegurar que app_ventas_user heredó el rol base

```
GRANT rol_usuario_base TO app_ventas_user;
```

Si el error no existe la relación «ventas.clientes» persiste incluso tras haber otorgado los permisos, el problema no es de permisos, sino de dónde se creó la tabla o de la sesión activa.

Aplica estos pasos para diagnosticar y corregir el problema:

1. Comprueba si la tabla realmente se creó en esa Base de Datos

En PostgreSQL, los esquemas y tablas son locales a cada Base de Datos. Si creaste la tabla en la base de datos `postgres` e intentas hacer la consulta conectado a otra base de datos (por ejemplo `ventas_db`), devolverá exactamente ese error.

Ejecuta esta consulta con tu usuario administrador para ver si la tabla existe en algún esquema:

SQL

```
SELECT table_catalog AS base_de_datos, table_schema, table_name
FROM information_schema.tables
WHERE table_name = 'clientes';
```

- Si la consulta devuelve 0 filas: La tabla nunca llegó a ejecutarse en la base de datos actual.
- Si la consulta muestra `public` en `table_schema`: La creaste en el esquema público por defecto sin el prefijo `ventas..`

2. Script Completo de Inicialización (Limpio)

Para asegurar que todo se cree en el orden correcto y en la base de datos adecuada, ejecuta todo este bloque desde la consola de administración (como usuario `postgres` o `superusuario`):

SQL

```
-- 1. Crear el esquema
CREATE SCHEMA IF NOT EXISTS ventas;

-- 2. Crear los roles si no existen
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname =
'rol_usuario_base') THEN
        CREATE ROLE rol_usuario_base NOLOGIN;
```

```

END IF;

IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname =
'app_ventas_user') THEN
    CREATE ROLE app_ventas_user WITH LOGIN PASSWORD 'PasswordSegura123!';
END IF;
END
$$;

-- 3. Crear las tablas dentro del esquema ventas
CREATE TABLE IF NOT EXISTS ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ventas.productos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL CHECK (precio >= 0),
    stock INT NOT NULL CHECK (stock >= 0)
);

CREATE TABLE IF NOT EXISTS ventas.pedidos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id INT NOT NULL REFERENCES ventas.clientes(id) ON DELETE RESTRICT,
    producto_id INT NOT NULL REFERENCES ventas.productos(id) ON DELETE RESTRICT,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Asignar permisos sobre el esquema, tablas y secuencias
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO
rol_usuario_base;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA ventas TO rol_usuario_base;
GRANT rol_usuario_base TO app_ventas_user;

```

3. Asignar permisos por defecto para futuras tablas

Si la tabla `clientes` fue creada por el usuario `postgres` después de otorgar el `GRANT`, las versiones de PostgreSQL previas o ciertas configuraciones no aplican retroactivamente el permiso a tablas nuevas a menos que se use `ALTER DEFAULT PRIVILEGES`:

SQL

```

ALTER DEFAULT PRIVILEGES IN SCHEMA ventas
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO rol_usuario_base;

ALTER DEFAULT PRIVILEGES IN SCHEMA ventas
GRANT USAGE, SELECT ON SEQUENCES TO rol_usuario_base;

```

4. Prueba de Verificación

Una vez ejecutado el script anterior, prueba la consulta cambiando al rol de aplicación:

SQL

```

-- Cambiar temporalmente de rol para probar
SET ROLE app_ventas_user;

-- Probar consulta explícita apuntando al esquema

```

```
SELECT * FROM ventas.clientes;
```

Si ejecutas `SELECT * FROM ventas.clientes;` tras aplicar el bloque anterior y continúa fallando, confirma con el comando `\C` (en psql) o en los detalles de tu cliente gráfico (DBeaver, pgAdmin) el nombre de la base de datos activa a la que estás conectado.

```
SELECT * FROM ventas.clientes;
```

 y no me devuelve nada

Si no devuelve ningún error y simplemente te muestra 0 filas (vacío), el comportamiento es exactamente el que diseñamos por Row-Level Security (RLS).

Cuando ejecutamos RLS en la tabla `ventas.clientes`, definimos que solo se muestren los registros donde la columna `region` coincida con la variable de sesión `app.app_region`. Si esa variable no se ha definido en la sesión actual, vale `NULL`, por lo que PostgreSQL oculta todas las filas por seguridad.

Diagnóstico Rápido

Ejecuta estas consultas en orden para entender qué está pasando:

1. Comprobar cuántos registros hay en realidad (como Administrador)

Si cambias a un rol que ignore RLS (como `postgres`), verás si hay datos insertados:

SQL

```
-- Restablecer al rol administrador de la sesión
RESET ROLE;
```

```
-- Consultar la tabla sin filtros de RLS
SELECT * FROM ventas.clientes;
```

2. Consultar como `app_ventas_user` definiendo la variable de región

Si estás conectado con el rol de la aplicación o usas `SET ROLE app_ventas_user;`, debes declarar la región antes de hacer la consulta:

SQL

```
SET ROLE app_ventas_user;
```

```
-- 1. Definir la región para esta sesión/transacción
SET LOCAL app.app_region = 'US';
```

```
-- 2. Ahora la consulta devolverá los clientes de esa región
SELECT * FROM ventas.clientes;
```

¿Por qué sucede esto?

Recuerda la política RLS que creamos:

SQL

```
CREATE POLICY policy_clientes_select ON ventas.clientes
  FOR SELECT
  TO rol_usuario_base
  USING (region = current_setting('app.app_region', true));
```

- `current_setting('app.app_region', true)` evalúa a `NULL` si no has ejecutado `SET app.app_region = 'US';` antes.

- La condición `region = NULL` en SQL resulta en UNKNOWN (falso), por lo que PostgreSQL filtra el 100% de las filas.

Flujo Completo de Prueba (Copia y Ejecuta)

Para probar insertar y consultar datos correctamente con RLS habilitado:

SQL

```
-- 1. Iniciar sesión como usuario de aplicación
SET ROLE app_ventas_user;

-- 2. Definir que la app está operando en la región 'US'
SET LOCAL app.app_region = 'US';

-- 3. Insertar un cliente (Solo funcionará si la región enviada coincide con 'US')
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');

-- 4. Consultar (Ahora devolverá a John Doe)
SELECT * FROM ventas.clientes;

-- 5. Cambiar la región a 'EU'
SET LOCAL app.app_region = 'EU';

-- 6. Al consultar ahora, volverá a salir 0 filas porque John Doe es de 'US'
SELECT * FROM ventas.clientes;
¿Quieres que ajustemos la política RLS para que muestre todos los registros si la variable
app_region no está definida?
```

Sí

he entrado como: `psql -h 127.0.0.1 -U app_ventas_user -d appdb`

Exacto, ese comportamiento es 100% normal y esperado con el diseño de Row-Level Security (RLS) que configuramos.

Como has iniciado sesión directamente con el usuario de la aplicación `app_ventas_user`, PostgreSQL le aplica las políticas RLS estrictamente. En este momento, la variable de sesión `app.app_region` aún no tiene ningún valor asignado (es NULL), por lo que la base de datos oculta todas las filas por seguridad.

Cómo trabajar desde esa terminal (psql)

Estando dentro de tu sesión actual con `app_ventas_user`, ejecuta los siguientes comandos paso a paso:

1. Establece la región en la sesión

SQL

```
SET app.app_region = 'US';
```

2. Inserta un cliente para esa región

SQL

```
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');
```

3. Realiza la consulta nuevamente

SQL

```
SELECT * FROM ventas.clientes;
```

Ahora sí verás la fila de John Doe.

4. Intenta insertar un cliente de otra región (Prueba de Seguridad)

SQL

```
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');
```

Lanzará un error de violación de política RLS porque tu sesión está fijada en 'US'.

5. Cambia de región a 'EU'

SQL

```
SET app.app_region = 'EU';
```

```
-- Insertar ahora a Pierre (Permitido)
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');
```

```
-- Consultar
SELECT * FROM ventas.clientes;
```

Verás únicamente a Pierre Dubois, mientras que John Doe quedará oculto.

Nota sobre SET vs SET LOCAL en psql:

- En la consola interactiva psql, usa `SET app.app_region = 'US';` para que el valor se mantenga durante toda tu sesión en la terminal.
- En el backend de una aplicación (Node.js, Python, Java, etc.), se utiliza `SET LOCAL app.app_region = 'US';` dentro de un bloque de transacción (`BEGIN ... COMMIT`) para evitar que la variable contamine otras peticiones en el pool de conexiones.

```
appdb=> SET app.app_region = 'US';
```

SET

```
appdb=> INSERT INTO ventas.clientes (nombre, email, region)
```

```
VALUES ('John Doe', 'john@example.com', 'US');
```

```
INSERT 0 1
```

```
appdb=> SELECT * FROM ventas.clientes;
```

```
id | nombre | email | region | fecha_registro
```

```
----+-----+-----+-----+-----
```

(0 filas)

El hecho de que el `INSERT` haya sido exitoso (`INSERT 0 1`) pero el `SELECT` devuelva 0 filas nos indica exactamente qué está sucediendo: el `INSERT` funcionó gracias a la política con `WITH CHECK`, pero el `SELECT` está fallando porque la política con `USING` no está evaluando `current_setting('app.app_region', true)` como esperábamos.

En PostgreSQL, `current_setting('app.app_region', true)` a veces requiere que la variable haya sido registrada formalmente o que el tipo de dato coincida de manera explícita (`::text`).

Solución en 2 Pasos

Paso 1: Recrear la política de `SELECT` con casteo explícito

Conéctate como superusuario (o con el rol creador) y actualiza la política para forzar el tipo de texto y manejar posibles espacios o comillas:

SQL

```
-- Eliminar la política anterior de SELECT
DROP POLICY IF EXISTS policy_clientes_select ON ventas.clientes;

-- Volver a crearla con casteo explícito a TEXT
CREATE POLICY policy_clientes_select ON ventas.clientes
    FOR SELECT
    TO rol_usuario_base
    USING (region::text = NULLIF(current_setting('app.app_region', true),
'')::text);
```

Y haz lo mismo con la política de `INSERT` para mantener consistencia:

SQL

```
DROP POLICY IF EXISTS policy_clientes_insert ON ventas.clientes;

CREATE POLICY policy_clientes_insert ON ventas.clientes
    FOR INSERT
    TO rol_usuario_base
    WITH CHECK (region::text = NULLIF(current_setting('app.app_region', true),
'')::text);
```

Paso 2: Probar desde tu sesión `app_ventas_user`

Regresa a la terminal donde estás conectado como `app_ventas_user` y ejecuta:

SQL

```
-- 1. Asignar la variable
SET app.app_region = 'US';

-- 2. Verificar qué valor está leyendo PostgreSQL exactamente
SELECT current_setting('app.app_region', true);

-- 3. Realizar el SELECT
SELECT * FROM ventas.clientes;
```

¿Por qué ocurrió esto?

1. En la política de `INSERT`, PostgreSQL evalúa la condición al momento de la inserción.
2. En la política de `SELECT`, si no hay una conversión explícita entre el tipo `VARCHAR(2)` de la columna `region` y el tipo devuelto por `current_setting()` (que es `text`), la comparación

`region = current_setting(...)` puede devolver UNKNOWN o evaluar en falso según el contexto de ejecución.

Al agregar `::text` y `NULLIF(..., '')`, aseguramos una comparación exacta entre strings.

como se asignan estas variables : 'Stock insuficiente para el producto ID %. Disponible: %, Solicitado: %',

En PostgreSQL (específicamente en PL/pgSQL), los comodines % en la orden RAISE EXCEPTION (o RAISE NOTICE, RAISE WARNING) funcionan como marcadores de posición que se reemplazan en orden por las variables o expresiones que pones separadas por comas al final de la sentencia.

Sintaxis y Ejemplo

La sintaxis básica es:

SQL

```
RAISE EXCEPTION 'mensaje con comodines %', variable1, variable2, variable3;
```

Para tu caso concreto, la asignación se hace pasando las variables exactamente en el mismo orden en el que aparecen los %:

SQL

```
RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible: %, Solicitado: %',  
    NEW.producto_id,  
    v_stock_actual,  
    NEW.cantidad;
```

¿Cómo se mapean?

1. El primer % se reemplaza por `NEW.producto_id`
2. El segundo % se reemplaza por `v_stock_actual`
3. El tercer % se reemplaza por `NEW.cantidad`

Ejemplo dentro de la función PL/pgSQL

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()  
RETURNS TRIGGER AS $$  
DECLARE  
    v_stock_actual INT := 5; -- Ejemplo: hay 5 unidades  
BEGIN  
    -- Supongamos que NEW.producto_id = 12 y NEW.cantidad = 10  
  
    IF v_stock_actual < NEW.cantidad THEN  
        RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:  
%, Solicitado: %',  
            NEW.producto_id,  
            v_stock_actual,  
            NEW.cantidad;  
    END IF;
```

```
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Salida generada al intentar insertar:

```
ERROR: Stock insuficiente para el producto ID 12.  
Disponible: 5, Solicitado: 10
```

Tip Adicional: Formato de Tipos

Si necesitas dar formato específico a un valor dentro del mensaje, puedes usar modificadores de formato con `format()`:

- `%o %S`: Imprime el valor como texto genérico (comportamiento por defecto de `RAISE`).
- `%I`: Se usa para formatear identificadores (nombres de tablas/columnas de forma segura).
- `%L`: Se usa para literales (añade comillas simples automáticamente y gestiona valores `NULL`).

en las nuevas políticas de pedidos habría que hacer también una conversión con la variable `region`?

Sí, es una excelente idea aplicarle la misma conversión `::text` a la variable en las políticas de `pedidos` por consistencia y para prevenir el mismo comportamiento que experimentaste con la tabla `clientes`.

Al realizar la comparación dentro de un subcomando `EXISTS`, PostgreSQL compara la columna `c.region` contra el resultado de `current_setting(...)`. Usar la misma función limpiadora `NULLIF(current_setting('app.app_region', true), '')::text` garantiza que el valor de la variable se interprete siempre como un `text` limpio.

Recreación de las Políticas para `ventas.pedidos`

Conéctate como superusuario (o rol propietario) y actualiza las políticas de la tabla `pedidos`:

SQL

```
-- 1. Asegurar que RLS esté activo en la tabla  
ALTER TABLE ventas.pedidos ENABLE ROW LEVEL SECURITY;  
  
-- 2. Eliminar políticas previas si existen  
DROP POLICY IF EXISTS policy_pedidos_insert ON ventas.pedidos;  
DROP POLICY IF EXISTS policy_pedidos_select ON ventas.pedidos;  
  
-- 3. Política para INSERT: Validar que el cliente pertenece a la región activa  
CREATE POLICY policy_pedidos_insert ON ventas.pedidos  
FOR INSERT  
TO rol_usuario_base  
WITH CHECK (  
    EXISTS (  
        SELECT 1  
        FROM ventas.clientes c  
        WHERE c.id = NEW.cliente_id  
              AND c.region::text = NULLIF(current_setting('app.app_region',  
true), '')::text  
    )  
);
```

```
-- 4. Política para SELECT: Permitir listar solo pedidos de la región activa
CREATE POLICY policy_pedidos_select ON ventas.pedidos
FOR SELECT
TO rol_usuario_base
USING (
    EXISTS (
        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
        AND c.region::text = NULLIF(current_setting('app.app_region',
true), '')::text
    )
);
```

Prueba rápida en la terminal (psql) con app_ventas_user

SQL

```
-- Configurar la sesión en región US
SET app.app_region = 'US';

-- Supongamos que John Doe (ID 1) es de 'US' y Pierre Dubois (ID 2) es de 'EU'

-- 1. Hacer pedido para John Doe (ID 1) estando en 'US' -> PERMITIDO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad) VALUES (1, 1, 2);

-- 2. Intentar hacer pedido para Pierre Dubois (ID 2) estando en 'US' ->
BLOQUEADO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad) VALUES (2, 1, 1);
-- Devuelve: ERROR: new row violates row-level security policy for table
"pedidos"

-- 3. Consultar los pedidos de la región 'US'
SELECT * FROM ventas.pedidos;

¿Quieres agregar también políticas RLS para UPDATE y DELETE en pedidos?
```

Funciones y Procedimientos

En esta guía práctica aprenderás a crear **Funciones y Procedimientos Almacenados** en PostgreSQL 18 utilizando el esquema ventas que hemos venido construyendo.

1. La Diferencia Clave: ¿Función o Procedimiento?

Característica	Función (CREATE FUNCTION)	Procedimiento (CREATE PROCEDURE)
Valor de retorno	Obligatorio. Siempre devuelve algo (un tipo básico, una tabla o un TRIGGER).	No devuelve valor directamente. Se usa para ejecutar acciones.

Característica	Función (CREATE FUNCTION)	Procedimiento (CREATE PROCEDURE)
Control de Transacciones	No permite COMMIT ni ROLLBACK dentro de su cuerpo. Es atómica.	Sí permite COMMIT y ROLLBACK internos (gestión manual de transacciones).
Forma de invocación	Se llama con SELECT: SELECT funcion();	Se llama con CALL: CALL procedimiento();

2. Creando Funciones (CREATE FUNCTION)

Las funciones son ideales para **calcular valores, consultar datos procesados o transformar información.**

Ejemplo 1: Función escalar simple (Calcular el Total de un Pedido)

Esta función recibe el precio de un producto y la cantidad solicitada, aplicando un descuento opcional.

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_calcular_total_pedido(
    pPrecio NUMERIC,
    pCantidad INT,
    pDescuentoPorcentaje NUMERIC DEFAULT 0
)
RETURNS NUMERIC AS $$
DECLARE
    vSubtotal NUMERIC;
    vTotal NUMERIC;
BEGIN
    -- 1. Operación básica
    vSubtotal := pPrecio * pCantidad;

    -- 2. Aplicar descuento
    vTotal := vSubtotal - (vSubtotal * (pDescuentoPorcentaje / 100));

    -- 3. Retornar el resultado
    RETURN vTotal;
END;
$$ LANGUAGE plpgsql;
```

Cómo probar la función:

SQL

```
-- Sin descuento (usa el valor por defecto 0)
SELECT ventas.fn_calcular_total_pedido(150.00, 2);
-- Devuelve: 300.00
```

```
-- Con un 10% de descuento
SELECT ventas.fn_calcular_total_pedido(150.00, 2, 10);
-- Devuelve: 270.00
```

Ejemplo 2: Función que devuelve una Tabla (RETURNS TABLE)

Imagina que quieres obtener un resumen de ventas de un cliente específico en una sola consulta estructurada.

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_resumen_compras_cliente(p_cliente_id INT)
RETURNS TABLE (
    pedido_id INT,
    producto_nombre VARCHAR,
    cantidad INT,
    fecha_pedido TIMESTAMP
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.id,
        prod.nombre,
        p.cantidad,
        p.fecha
    FROM ventas.pedidos p
    JOIN ventas.productos prod ON p.producto_id = prod.id
    WHERE p.cliente_id = p_cliente_id;
END;
$$ LANGUAGE plpgsql;
```

Cómo probar la función:

SQL

```
SELECT * FROM ventas.fn_resumen_compras_cliente(1);
```

3. Creando Procedimientos Almacenados (CREATE PROCEDURE)

Los procedimientos almacenados se utilizan para **ejecutar flujos de trabajo complejas** (procesar datos, insertar en múltiples tablas, reponer stock masivo) donde no necesitas una respuesta inmediata en filas, sino realizar cambios en el estado de la base de datos.

Ejemplo 3: Procedimiento de Reabastecimiento de Inventario

Crearemos un procedimiento para reponer el stock de un producto y registrar la acción en una tabla de auditoría.

Paso previo: Crear la tabla de auditoría

SQL

```
CREATE TABLE IF NOT EXISTS ventas.historial_reabastecimiento (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    producto_id INT REFERENCES ventas.productos(id),
    cantidad_agregada INT NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Creación del Procedimiento:

SQL

```
CREATE OR REPLACE PROCEDURE ventas.pr_reabastecer_producto(
    p_producto_id INT,
    p_cantidad INT
)
LANGUAGE plpgsql AS $$
BEGIN
    -- Validar que la cantidad a reponer sea lógica
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad a reponer debe ser mayor a 0. Solicitado: %',
p_cantidad;
    END IF;

    -- 1. Incrementar el stock del producto
    UPDATE ventas.productos
    SET stock = stock + p_cantidad
    WHERE id = p_producto_id;

    -- Validar si el producto realmente existía
    IF NOT FOUND THEN
        RAISE EXCEPTION 'No se encontró el producto con ID %', p_producto_id;
    END IF;

    -- 2. Registrar el movimiento en el historial
    INSERT INTO ventas.historial_reabastecimiento (producto_id, cantidad_agregada)
    VALUES (p_producto_id, p_cantidad);

    -- Confirmación explícita (opcional, PostgreSQL lo hace automáticamente al terminar la
transacción)
    COMMIT;
END;
$$;
```

Cómo probar el procedimiento:

SQL

```
-- Ejecutar la llamada con CALL
CALL ventas.pr_reabastecer_producto(1, 50);

-- Verificar el incremento en el producto
SELECT id, nombre, stock FROM ventas.productos WHERE id = 1;

-- Verificar la entrada en el historial
SELECT * FROM ventas.historial_reabastecimiento;
```

4. Conceptos Clave de PL/pgSQL para recordar

1. **Variables:** Se declaran en el bloque DECLARE con su tipo (v_total NUMERIC;).
2. **Asignación:** Se utiliza el operador := para asignar valores a las variables (v_subtotal := p_precio * p_cantidad;).
3. **Estructuras de control:** Puedes usar bloques condicionales IF ... THEN ... ELSE ... END IF; o bucles LOOP / WHILE.
4. **FOUND:** Es una variable booleana especial de PL/pgSQL que se vuelve TRUE si la última sentencia SQL (UPDATE, DELETE, SELECT INTO) afectó al menos una fila.
- 5.

Bloques EXCEPTION (TRY/CATCH)

En PL/pgSQL, la gestión de errores tipo TRY/CATCH se implementa mediante un bloque **BEGIN ... EXCEPTION ... END.**

Cuando ocurre un error dentro de la sección **BEGIN**, la ejecución se interrumpe inmediatamente y el control pasa a la sección **EXCEPTION**. En ese punto, la subtransacción dentro del bloque hace un **ROLLBACK automático de los cambios ocurridos dentro de ese BEGIN**, permitiéndote capturar la anomalía, registrar un mensaje personalizado o manejar el fallo sin detener toda la aplicación.

Sintaxis Estructurada

SQL

```
BEGIN
  -- [Sección TRY]
  -- Código que podría fallar (INSERT, UPDATE, divisiones, etc.)
EXCEPTION
  -- [Sección CATCH]
  WHEN tipo_de_error THEN
    -- Acciones al capturar un error específico
  WHEN OTHERS THEN
    -- Acciones para cualquier otro error no especificado
END;
```

Ejemplo Práctico: Registro Seguro de Pedidos con Manejo de Excepciones

Crearemos un procedimiento almacenado para la tabla `ventas.pedidos` que intente procesar una compra. Si el producto no tiene suficiente stock o no existe (lanzando el error desde nuestro *trigger* o *foreign key*), el procedimiento **capturará el error**, registrará el fallo en una tabla de auditoría de errores y evitará que la aplicación colapse.

1. Tabla para Registrar los Errores (Log de Excepciones)

SQL

```
CREATE TABLE IF NOT EXISTS ventas.log_errores_pedidos (
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  cliente_id INT,
```



```

    producto_id INT,
    cantidad INT,
    codigo_error TEXT,
    mensaje_error TEXT,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

2. Procedimiento Almacenado con Bloque EXCEPTION

Usaremos variables especiales de PL/pgSQL como SQLSTATE (el código de error numérico/alfanumérico) y SQLERRM (el mensaje del error) a través de la cláusula GET STACKED DIAGNOSTICS.

SQL

```

CREATE OR REPLACE PROCEDURE ventas.pr_registrar_pedido_seguro(
    p_cliente_id INT,
    p_producto_id INT,
    p_cantidad INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_sqlstate TEXT;
    v_sqlerrm TEXT;
BEGIN
    -----
    -- BLOQUE TRY: Intentamos realizar la inserción
    -----
    BEGIN
        INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
        VALUES (p_cliente_id, p_producto_id, p_cantidad);

        RAISE NOTICE 'Pedido registrado exitosamente para el cliente %',
p_cliente_id;

        -----
        -- BLOQUE CATCH: Si ocurre algún error dentro del BEGIN anterior
        -----
    EXCEPTION
        -- Capturar errores específicos de Check Constraints o FKs
        WHEN foreign_key_violation THEN
            GET STACKED DIAGNOSTICS v_sqlstate = RETURNED_SQLSTATE, v_sqlerrm =
MESSAGE_TEXT;

            INSERT INTO ventas.log_errores_pedidos (cliente_id, producto_id,
cantidad, codigo_error, mensaje_error)
            VALUES (p_cliente_id, p_producto_id, p_cantidad, v_sqlstate,
'Violación de clave foránea: El cliente o producto no existe.');
```

RAISE NOTICE 'Transacción fallida (FK Violation): Registrado en log_errores_pedidos.';

```

        -- Capturar excepciones personalizadas lanzadas por Triggers (como stock
insuficiente)
        WHEN raise_exception THEN
            GET STACKED DIAGNOSTICS v_sqlstate = RETURNED_SQLSTATE, v_sqlerrm =
MESSAGE_TEXT;

            INSERT INTO ventas.log_errores_pedidos (cliente_id, producto_id,
cantidad, codigo_error, mensaje_error)
            VALUES (p_cliente_id, p_producto_id, p_cantidad, v_sqlstate,
v_sqlerrm);

```

```

        RAISE NOTICE 'Transacción fallida (Excepción de negocio): %',
v_sqlerrm;

        -- Capturar cualquier otro error imprevisto
        WHEN OTHERS THEN
            GET STACKED DIAGNOSTICS v_sqlstate = RETURNED_SQLSTATE, v_sqlerrm =
MESSAGE_TEXT;

            INSERT INTO ventas.log_errores_pedidos (cliente_id, producto_id,
cantidad, codigo_error, mensaje_error)
            VALUES (p_cliente_id, p_producto_id, p_cantidad, v_sqlstate,
v_sqlerrm);

            RAISE NOTICE 'Error no esperado capturado: % (SQLSTATE %)',
v_sqlerrm, v_sqlstate;
        END;
    END;
$$;

```

Pruebas de Funcionamiento

Caso 1: Insertar un pedido válido (Stock suficiente)

SQL

```

-- Supongamos que Cliente 1 y Producto 1 existen y tienen stock
CALL ventas.pr_registrar_pedido_seguro(1, 1, 2);
-- NOTICE: Pedido registrado exitosamente para el cliente 1

```

Caso 2: Forzar un error de Stock Insuficiente (Capturado por WHEN raise_exception)

SQL

```

-- Intentamos pedir 9999 unidades del producto 1
CALL ventas.pr_registrar_pedido_seguro(1, 1, 9999);
-- NOTICE: Transacción fallida (Excepción de negocio): Stock insuficiente para
el producto ID 1...

```

Caso 3: Forzar un error de Cliente Inexistente (Capturado por WHEN foreign_key_violation)

SQL

```

-- El cliente ID 99999 no existe en la BD
CALL ventas.pr_registrar_pedido_seguro(99999, 1, 1);
-- NOTICE: Transacción fallida (FK Violation): Registrado en
log_errores_pedidos.

```

Comprobar la Tabla de Log

SQL

```

SELECT * FROM ventas.log_errores_pedidos;

```

Puntos Clave a Tener en Cuenta

1. **Rendimiento:** Crear bloques `BEGIN . . . EXCEPTION` genera un subpunto de restauración (*savepoint*) interno en PostgreSQL. Evita envolver miles de inserciones en un bucle individual con `EXCEPTION` si buscas alto rendimiento de escritura masiva.
2. **Re-lanzar errores (RAISE):** Si quieres registrar el error en el log pero **también** hacer que la aplicación cliente reciba la falla, puedes volver a lanzar el error dentro del bloque `EXCEPTION` usando la instrucción `RAISE`; sin argumentos:

SQL

```
EXCEPTION
    WHEN OTHERS THEN
        INSERT INTO ventas.log_errores_pedidos (...) VALUES (...);
        RAISE; -- Vuelve a propagar la excepción hacia afuera
```

¿Quieres explorar más patrones avanzados en PL/pgSQL?

Ver cómo usar cursores para procesar pedidos de forma iterativa

Aprender sobre transacciones explícitas (`COMMIT/ROLLBACK`) en procedimientos

Cursores en PL/pgSQL

Un cursor en PL/pgSQL es un puntero o mecanismo que te permite recorrer e inspeccionar un conjunto de resultados fila por fila.

A diferencia de una consulta `SELECT` estándar que procesa y devuelve todas las filas de golpe en memoria, un cursor extrae un registro a la vez (*o en bloques*). Esto resulta especialmente útil para **algoritmos complejos de procesamiento iterativo**, como aplicar reglas de negocio dinámicas, calcular acumulados fila por fila o enviar notificaciones registro por registro.

Ciclo de vida de un Cursor en PL/pgSQL

Para trabajar con un cursor de forma manual se siguen cuatro pasos:

1. **Declaración:** Se define la consulta `SELECT` asociada al cursor.
2. **Apertura (OPEN):** Se ejecuta la consulta y se inicializa la posición del puntero.
3. **Extracción (FETCH):** Se lee el registro actual y se mueven los datos a variables locales.
4. **Cierre (CLOSE):** Se liberan los recursos asociados al cursor.

Ejemplo Práctico: Procesamiento Nocturno de Pedidos y Evaluación de Clientes

Imagina que queremos procesar los pedidos recientes para aplicar un **descuento VIP del 10%** en compras futuras si un pedido supera los \$500, o registrar una **alerta de bajo valor** si no llega a \$20.

1. Estructura de la función con Cursor Explícito

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_procesar_evaluacion_pedidos()
RETURNS VOID AS $$
DECLARE
    -- 1. DECLARACIÓN del cursor y la variable que detectará el fin de filas
    cur_pedidos CURSOR FOR
        SELECT p.id, p.cliente_id, prod.nombre, p.cantidad, prod.precio
        FROM ventas.pedidos p
        JOIN ventas.productos prod ON p.producto_id = prod.id;

    v_pedido RECORD; -- Variable de tipo registro para guardar la fila actual
    v_total NUMERIC;
BEGIN
    -- 2. APERTURA del cursor
    OPEN cur_pedidos;

    LOOP
        -- 3. EXTRAER la fila actual dentro de la variable de registro
        FETCH cur_pedidos INTO v_pedido;

        -- Condición de salida: Cuando no haya más filas para leer
        EXIT WHEN NOT FOUND;

        -- Calcular total de la fila actual
        v_total := v_pedido.cantidad * v_pedido.precio;

        -- Lógica de negocio fila por fila
        IF v_total > 500.00 THEN
            RAISE NOTICE '¡Pedido VIP! ID: %, Cliente: %, Producto: %, Total: $
%',
                v_pedido.id, v_pedido.cliente_id, v_pedido.nombre, v_total;
        ELSIF v_total < 20.00 THEN
            RAISE NOTICE 'Pedido Pequeño. ID: %, Total: $%',
                v_pedido.id, v_total;
        ELSE
            RAISE NOTICE 'Pedido Estándar. ID: %, Total: $%',
                v_pedido.id, v_total;
        END IF;

    END LOOP;

    -- 4. CIERRE del cursor
    CLOSE cur_pedidos;
END;
$$ LANGUAGE plpgsql;
```

La Forma Moderna y Simplificada: Bucle FOR sobre Cursor

En la práctica moderna de PostgreSQL, rara vez necesitas hacer OPEN, FETCH y CLOSE manualmente. Puedes usar un bucle **FOR ... IN**, donde PL/pgSQL gestiona automáticamente la apertura, lectura, condición de parada y cierre del cursor por ti:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_procesar_pedidos_simplificado()
RETURNS VOID AS $$
DECLARE
    v_total NUMERIC;
BEGIN
```

```

-- El bucle FOR abre el cursor implícitamente y crea la variable 'rec'
FOR rec IN
    SELECT p.id, p.cliente_id, prod.nombre, p.cantidad, prod.precio
    FROM ventas.pedidos p
    JOIN ventas.productos prod ON p.producto_id = prod.id
LOOP
    v_total := rec.cantidad * rec.precio;

    IF v_total > 500.00 THEN
        RAISE NOTICE 'VIP ID: %, Total: $%', rec.id, v_total;
    END IF;
END LOOP;
-- Al terminar el bucle, PostgreSQL cierra el cursor automáticamente
END;
$$ LANGUAGE plpgsql;

```

OTRO EJEMPLO:

- procedimiento que lee cada pedido e inserta en otra tabla el numero del pedido y el total.

SQL

```

CREATE TABLE IF NOT EXISTS ventas.resumen (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pedido_id INT NOT NULL,
    total NUMERIC(12, 2) NOT NULL,
    fecha_procesamiento TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    -- Definición de la clave foránea
    CONSTRAINT fk_resumen_pedidos
        FOREIGN KEY (pedido_id)
        REFERENCES ventas.pedidos (id)
        ON DELETE CASCADE
);

```

Puntos clave de esta estructura:

- **id BIGINT GENERATED ALWAYS AS IDENTITY:** Es la forma estándar ANSI SQL recomendada en versiones modernas de PostgreSQL para claves primarias autoincrementables (reemplaza al tipo SERIAL).
- **FOREIGN KEY (pedido_id):** Garantiza la integridad referencial conectando directamente con la tabla de pedidos.
- **ON DELETE CASCADE:** Si se elimina un pedido en `ventas.pedidos`, sus registros asociados en la tabla de resumen se borran automáticamente para evitar datos huérfanos.

Procedimiento actualizado para esta nueva estructura

Dado que ahora el `pedido_id` ya no es la clave primaria de esta tabla, el procedimiento para poblarla cambia ligeramente (usando `WHERE NOT EXISTS` para evitar duplicar resúmenes del mismo pedido):

SQL

```
CREATE OR REPLACE PROCEDURE ventas.pr_generar_resumen()
LANGUAGE plpgsql AS $$
DECLARE
    rec RECORD;
    v_total NUMERIC(12, 2);
    v_procesados INT := 0;
BEGIN
    FOR rec IN
        SELECT
            p.id AS pedido_id,
            p.cantidad,
            prod.precio
        FROM ventas.pedidos p
        JOIN ventas.productos prod ON p.producto_id = prod.id
    LOOP
        v_total := rec.cantidad * rec.precio;

        -- Insertar solo si el pedido aún no tiene un registro en la tabla
        resumen
        INSERT INTO ventas.resumen (pedido_id, total)
        SELECT rec.pedido_id, v_total
        WHERE NOT EXISTS (
            SELECT 1 FROM ventas.resumen r WHERE r.pedido_id = rec.pedido_id
        );

        v_procesados := v_procesados + 1;
    END LOOP;

    RAISE NOTICE 'Proceso finalizado. Filas evaluadas: %', v_procesados;
END;
$$;
```

Explicación rápida de las partes clave:

- **rec RECORD:** Variable tipo registro donde se carga la fila actual de la consulta en cada iteración del bucle.
- **v_total:** Variable local (v_) donde almacenamos temporalmente el resultado de `cantidad * precio`.
- **WHERE NOT EXISTS (SELECT 1 FROM ventas.resumen r WHERE r.pedido_id = rec.pedido_id);**

que significa esto: `WHERE NOT EXISTS (`

```
SELECT 1 FROM ventas.resumen r WHERE r.pedido_id = rec.pedido_id
);
```

Esa subconsulta con **WHERE NOT EXISTS** es un mecanismo de control para **evitar insertar registros duplicados**.

Significa literalmente: **"Inserta la fila SOLO SI NO EXISTE ya un registro en la tabla ventas.resumen para este mismo pedido"**.

Desglose paso a paso de cómo funciona:

1. **rec.pedido_id**: Es el ID del pedido que el bucle FOR está procesando en ese instante.
2. **SELECT 1 FROM ventas.resumen r WHERE r.pedido_id = rec.pedido_id**:
 - Busca en la tabla `ventas.resumen` si ya hay alguna fila guardada con ese `pedido_id`.
 - El **SELECT 1** se usa por rendimiento: a PostgreSQL no le importa qué columnas devolver, solo quiere saber si **encuentra al menos una fila que cumpla la condición**.
3. **NOT EXISTS (. . .)**:
 - Si la subconsulta **NO encuentra filas** (devuelve Falso) → **NOT EXISTS** se convierte en **Verdadero** → El **INSERT se ejecuta**.
 - Si la subconsulta **SÍ encuentra una fila** (devuelve Verdadero) → **NOT EXISTS** se convierte en **Falso** → El **INSERT se ignora silenciosamente** para ese pedido.

¿Por qué lo usamos aquí?

Dado que en la tabla `ventas.resumen` la clave primaria (**PRIMARY KEY**) es la columna `id` y **no** `pedido_id`, la base de datos por sí sola permitiría insertar el mismo pedido 10 veces si ejecutas el procedimiento 10 veces.

Con este **WHERE NOT EXISTS** garantizas que el procedimiento sea **idempotente**: puedes ejecutar `CALL ventas.pr_generar_resumen()`; cuantas veces quieras, y solo procesará e insertará los pedidos nuevos que no se hayan registrado previamente.

rec.cantidad es igual a la variable p.cantidd?

Sí, exactamente. Es el mismo valor.

En la consulta del bucle FOR:

SQL

```
FOR rec IN
  SELECT
    p.id AS pedido_id,
    p.cantidad,          -- <--- Aquí lees la columna de la tabla
ventas.pedidos (p)
    prod.precio
  FROM ventas.pedidos p
  JOIN ventas.productos prod ON p.producto_id = prod.id
```

`p.cantidad` es el nombre que tiene la columna dentro del **SELECT**. Cuando el bucle **FOR** asigna esa fila a la variable `rec` (que es de tipo **RECORD**), puedes acceder a sus campos usando la sintaxis `rec.nombre_columna`.

Por lo tanto:

- `p.cantidad` hace referencia a la columna `cantidad` de la tabla `pedidos` (con alias `p`) en la consulta SQL.

- `rec.cantidad` es cómo lees ese mismo valor dentro del código PL/pgSQL en cada iteración del bucle.

Para probarlo ejecuta:

SQL

```
CALL ventas.pr_generar_resumen();
SELECT * FROM ventas.resumen;
```

Modificar filas MIENTRAS recorres el Cursor (WHERE CURRENT OF)

Una de las ventajas más potentes de los cursores explícitos es la capacidad de **actualizar o eliminar la fila exacta sobre la que estás parado** usando la cláusula `WHERE CURRENT OF`:

SQL

```
CREATE OR REPLACE PROCEDURE ventas.pr_marcar_pedidos_revisados()
LANGUAGE plpgsql AS $$
DECLARE
    -- Definir un cursor con bloqueo 'FOR UPDATE'
    cur_pedidos_pendientes CURSOR FOR
        SELECT id, cantidad
        FROM ventas.pedidos
        WHERE cantidad > 0
        FOR UPDATE; -- Bloquea las filas seleccionadas

    v_rec RECORD;
BEGIN
    OPEN cur_pedidos_pendientes;

    LOOP
        FETCH cur_pedidos_pendientes INTO v_rec;
        EXIT WHEN NOT FOUND;

        -- Actualizar EXCLUSIVAMENTE la fila sobre la que el cursor apunta
        actualmente
        /*
            UPDATE ventas.pedidos
            SET procesado = TRUE
            WHERE CURRENT OF cur_pedidos_pendientes;
        */

        RAISE NOTICE 'Procesado pedido %', v_rec.id;
    END LOOP;

    CLOSE cur_pedidos_pendientes;
END;
$$;
```

Resumen: ¿Cuándo usar y cuándo NO usar Cursores?

- **SÍ usarlos cuando:**

- Necesites integrar lógica imperativa compleja por cada fila que no se puede resolver eficientemente con una sola consulta SQL declarativa.
- Proceses grandes cantidades de datos batch donde mantener millones de filas en memoria afectaría el rendimiento del servidor.
- **NO usarlos cuando:**
 - Puedas resolver el problema con un UPDATE, INSERT ... SELECT o DELETE masivo orientado a conjuntos (*Set-based SQL*). Las operaciones declarativas de SQL son significativamente más rápidas que los bucles en PL/pgSQL.

¿Cómo quieres profundizar en los conceptos de PostgreSQL?

Aprender sobre transacciones explícitas (COMMIT/ROLLBACK) dentro de procedimientos almacenados

Ver cómo crear funciones agregadas personalizadas o trabajar con arrays

Transacciones explícitas con COMMIT y ROLLBACK dentro de un procedimiento almacenado en PostgreSQL 18.

Una de las diferencias fundamentales entre las funciones (CREATE FUNCTION) y los procedimientos almacenados (CREATE PROCEDURE) en PostgreSQL es que los procedimientos sí permiten gestionar transacciones de forma explícita usando COMMIT y ROLLBACK dentro de su código.

Esto es extremadamente útil cuando necesitas procesar grandes volúmenes de datos en lotes (*batches*) o ejecutar operaciones independientes donde no quieres que el fallo de un paso cancele todo el trabajo previo.

Reglas Clave de las Transacciones en Procedimientos

1. **No se pueden usar dentro de un bloque BEGIN ... EXCEPTION:** Si un bloque de código captura excepciones con EXCEPTION, PostgreSQL crea internamente un *savepoint*, lo que impide ejecutar un COMMIT o ROLLBACK explícito dentro de ese bloque específico.
2. **COMMIT libera bloqueos:** Cuando ejecutas COMMIT, se persisten los datos procesados hasta ese punto y se liberan los bloqueos de filas/tablas.
3. **ROLLBACK deshace hasta el último punto de control:** Revierte todos los cambios realizados desde el inicio del procedimiento o desde el último COMMIT.

Ejemplo Práctico: Procesamiento de Lotes de Reposición con Guardado Parcial

Imagina que queremos reponer el stock de varios productos. Si uno de los productos falla (por ejemplo, si se intenta enviar una cantidad inválida), queremos **revertir solo esa operación**, pero **mantener guardadas (COMMIT) las reposiciones exitosas anteriores**.

1. Creación del Procedimiento con COMMIT y ROLLBACK

SQL

```
CREATE OR REPLACE PROCEDURE ventas.pr_reabastecer_lote(
    p_productos_ids INT[],
    p_cantidades INT[]
)
LANGUAGE plpgsql AS $$
DECLARE
    v_i INT;
    v_prod_id INT;
    v_cant INT;
BEGIN
    -- Validar que ambos arrays tengan el mismo tamaño
    IF array_length(p_productos_ids, 1) IS DISTINCT FROM array_length(p_cantidades, 1) THEN
        RAISE EXCEPTION 'Los arrays de IDs y cantidades deben tener la misma longitud.';
    END IF;

    -- Recorrer los elementos
    FOR v_i IN 1 .. array_length(p_productos_ids, 1) LOOP
        v_prod_id := p_productos_ids[v_i];
        v_cant := p_cantidades[v_i];

        RAISE NOTICE 'Procesando producto ID % con cantidad %...', v_prod_id, v_cant;

        -- Validación manual de negocio
        IF v_cant <= 0 THEN
            RAISE WARNING 'Cantidad inválida (%) para el producto ID %. Cancelando este
paso...', v_cant, v_prod_id;

            -- Revertir los cambios no confirmados de este ciclo
            ROLLBACK;

            -- Continuar con el siguiente producto del bucle
            CONTINUE;
        END IF;

        -- Actualizar stock
        UPDATE ventas.productos
        SET stock = stock + v_cant
        WHERE id = v_prod_id;

        IF NOT FOUND THEN
            RAISE WARNING 'Producto ID % no existe. Cancelando paso...', v_prod_id;
            ROLLBACK;
            CONTINUE;
        END IF;

        -- Confirmar los cambios de este producto en la base de datos de forma permanente
        COMMIT;
        RAISE NOTICE 'Éxito: Producto ID % actualizado y confirmando transacción
(COMMIT).', v_prod_id;
```

```
END LOOP;  
END;  
$$;
```

Demostración de Uso y Comprobación

Supongamos que tenemos dos productos en la tabla ventas.productos:

- **ID 1 (Laptop):** Stock actual = 10
- **ID 2 (Teclado):** Stock actual = 5

Ejecutamos el procedimiento enviando 3 operaciones en lote:

1. Agregar **20** unidades al Producto **ID 1** (*Válido*).
2. Agregar **-5** unidades al Producto **ID 2** (*Inválido: provoca ROLLBACK de esta iteración*).
3. Agregar **15** unidades al Producto **ID 2** (*Válido*).

SQL

```
CALL ventas.pr_reabastecer_lote(  
    ARRAY[1, 2, 2],  
    ARRAY[20, -5, 15]  
);
```

Salida de la Consola (RAISE NOTICE):

Plaintext

```
NOTICE: Procesando producto ID 1 con cantidad 20...  
NOTICE: Éxito: Producto ID 1 actualizado y confirmando transacción (COMMIT).  
NOTICE: Procesando producto ID 2 con cantidad -5...  
WARNING: Cantidad inválida (-5) para el producto ID 2. Cancelando este paso...  
NOTICE: Procesando producto ID 2 con cantidad 15...  
NOTICE: Éxito: Producto ID 2 actualizado y confirmando transacción (COMMIT).
```

Resultado Final en la Tabla:

SQL

```
SELECT id, nombre, stock FROM ventas.productos;
```

i d	nombre	stock
1	Laptop Pro 16	30 (<i>Subió de 10 a 30 por el primer COMMIT</i>)
2	Teclado Mecánico	20 (<i>Omitió el -5 por el ROLLBACK y subió de 5 a 20 en el segundo COMMIT</i>)

¿Cuándo usar Transacciones Explícitas en Procedimientos?

- **Procesamiento por Lotes (Batch Jobs):** Cuando actualizas o procesas 1,000,000 de registros y ejecutas un COMMIT cada 10,000 filas para

evitar saturar el log de WAL (*Write-Ahead Logging*) y reducir el consumo de memoria.

- **Integraciones y ETLs:** Cuando deseas registrar el progreso de la ejecución en una tabla de control/log incluso si el proceso principal falla.

Arrays y bloques FOREACH dentro de funciones PL/pgSQL en PostgreSQL.

En PostgreSQL, los **Arrays** (arreglos) son tipos de datos nativos que permiten almacenar listas de elementos (números, textos, fechas, etc.) en una sola columna o variable.

PL/pgSQL ofrece la estructura **FOREACH . . . IN ARRAY**, que está optimizada específicamente para iterar elemento por elemento a través de un array de forma limpia y legible, sin necesidad de gestionar índices manualmente.

1. Conceptos Básicos de Arrays en PL/pgSQL

- **Sintaxis de construcción:** Puedes definir arrays usando la palabra clave `ARRAY[ . . . ]` o literales entre comillas `'{"elem1", "elem2"}'`.
- **Indexación:** En PostgreSQL, los arrays por defecto **empiezan en el índice 1** (no en 0).
- **Funciones útiles nativas:**
 - `array_length(mi_array, 1)`: Devuelve la cantidad de elementos en la primera dimensión.
 - `array_append(mi_array, nuevo_elemento)` o el operador `mi_array || nuevo_elemento`: Añaden un elemento al final.

2. La Estructura FOREACH

A diferencia de un bucle `FOR` tradicional que itera sobre un rango de números (`FOR i IN 1..10`), el bucle `FOREACH` extrae **directamente cada valor** del array en una variable temporal.

Sintaxis Básica:

SQL

```
FOREACH variable_elem IN ARRAY mi_array_var LOOP
    -- Código a ejecutar por cada elemento
END LOOP;
```

3. Ejemplo Práctico: Descuento Masivo por Categorías de Productos

Imagina que queremos crear una función que reciba una lista de IDs de productos (un array `INT[]`) y les aplique un porcentaje de descuento individualmente, retornando una tabla con el resumen de los cambios.

Creación de la Función:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_aplicar_descuento_masivo(
    p_productos_ids INT[],
    p_porcentaje_descuento NUMERIC
)
RETURNS TABLE (
    producto_id INT,
    nombre_producto VARCHAR,
    precio_original NUMERIC,
    precio_con_descuento NUMERIC
) AS $$
DECLARE
    v_id INT; -- Variable temporal que almacenará cada elemento del array
    v_prod RECORD;
BEGIN
    -- Validar que el porcentaje sea válido
    IF p_porcentaje_descuento < 0 OR p_porcentaje_descuento > 100 THEN
        RAISE EXCEPTION 'El porcentaje de descuento debe estar entre 0 y 100.';
    END IF;

    -- Iterar elemento por elemento usando FOREACH
    FOREACH v_id IN ARRAY p_productos_ids LOOP

        -- Buscar y actualizar el precio del producto actual
        UPDATE ventas.productos
        SET precio = precio - (precio * (p_porcentaje_descuento / 100.0))
        WHERE id = v_id
        RETURNING id, nombre, (precio + (precio * (p_porcentaje_descuento /
100.0))), precio
        INTO v_prod;

        -- Si el producto existía, asignar los valores a las columnas de retorno
        IF FOUND THEN
            producto_id := v_prod.id;
            nombre_producto := v_prod.nombre;
            precio_original := v_prod.3; -- Tercer valor del RETURNING
            precio_con_descuento := v_prod.precio;

            RETURN NEXT; -- Emitir una fila hacia la tabla de retorno
        ELSE
            RAISE NOTICE 'El producto con ID % no fue encontrado.', v_id;
        END IF;

    END LOOP;

END;
$$ LANGUAGE plpgsql;
```

4. Bucle FOREACH con Arrays Multidimensionales (SLICE)

Si trabajas con matrices o arrays multidimensionales (por ejemplo, una lista de pares `[[cliente_id, producto_id], [cliente_id, producto_id]]`), puedes usar la cláusula `SLICE`.

`SLICE 1` le indica a PostgreSQL que en cada iteración no extraiga un elemento individual, sino un **sub-array de 1 dimensión**:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_procesar_matriz_pedidos(
    p_pedidos INT[][] -- Array 2D: ej. ARRAY[[1, 10], [2, 5]] -> [[cliente,
cantidad]]
)
RETURNS VOID AS $$
DECLARE
    v_par INT[]; -- Recibirá cada sub-array [cliente, cantidad]
BEGIN
    -- SLICE 1 toma filas enteras del array multidimensional
    FOREACH v_par SLICE 1 IN ARRAY p_pedidos LOOP

        RAISE NOTICE 'Procesando Cliente ID: %, Cantidad: %',
            v_par[1], v_par[2];

    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

5. Ejemplo de Prueba de las Funciones

SQL

```
-- 1. Probar descuento masivo en los productos ID 1 y 2
SELECT * FROM ventas.fn_aplicar_descuento_masivo(ARRAY[1, 2], 15.0);

-- 2. Probar la función multidimensional enviando pares [cliente, cantidad]
SELECT ventas.fn_procesar_matriz_pedidos(ARRAY[[1, 10], [2, 25], [3, 5]]);
```

Salida en consola para la prueba multidimensional:

Plaintext

```
NOTICE: Procesando Cliente ID: 1, Cantidad: 10
NOTICE: Procesando Cliente ID: 2, Cantidad: 25
NOTICE: Procesando Cliente ID: 3, Cantidad: 5
```

Resumen de Buenas Prácticas

1. Usa **FOREACH** para legibilidad: Es preferible a un `FOR i IN 1..array_length()` porque evita errores de índice fuera de rango (*off-by-one errors*).
2. Combina con **UNNEST** cuando busques velocidad: Si solo necesitas hacer un `JOIN` o un `IN` con un array en una consulta SQL directa, la función nativa `UNNEST(mi_array)` suele ser más rápida que recorrer el array en un bucle PL/pgSQL.
- 3.

usar UNNEST y los operadores de Arrays (@)

En PostgreSQL, trabajar con **Arrays** no se limita a recorrerlos con bucles en PL/pgSQL. El motor incluye potentes **operadores de conjuntos** y la función especial **UNNEST()**, que permiten manipular arreglos directamente dentro de consultas SQL declarativas (SELECT, JOIN, WHERE), obteniendo un rendimiento infinitamente superior.

1. La Función UNNEST()

La función UNNEST() toma un Array (una lista de elementos dentro de una sola celda) y lo **expande en múltiples filas** (una tabla temporal).

Ejemplo Básico

SQL

```
SELECT UNNEST(ARRAY['Laptop', 'Teclado', 'Mouse']) AS producto;
```

Resultado:

producto
Laptop
Teclado
Mouse

Caso de Uso Práctico: Inserción Masiva o Filtrado en ventas

Imagina que recibes un listado de IDs de productos en la aplicación y quieres consultar sus detalles sin necesidad de concatenar texto dinámico o hacer múltiples consultas:

SQL

```
-- Consultar productos cuyos IDs están en una lista
SELECT *
FROM ventas.productos
WHERE id IN (
    SELECT UNNEST(ARRAY[1, 2, 5])
);
```

Expandir múltiples arrays a la vez con WITH ORDINALITY

WITH ORDINALITY agrega una columna numerada que preserva la posición original de cada elemento:

SQL

```
SELECT *
FROM UNNEST(
    ARRAY['Laptop Pro', 'Teclado Mecánico'],
    ARRAY[1500.00, 120.00]
```

) WITH ORDINALITY AS t(nombre, precio, posicion);

Resultado:

nombre	precio	posicion
Laptop Pro	1500.00	1
Teclado Mecánico	120.00	2

2. Operadores Especiales de Arrays

PostgreSQL ofrece operadores específicos para comparar y evaluar relaciones entre arrays.

A. Operador @> (Contiene / Contains)

Evalúa si el array de la izquierda **contiene todos** los elementos del array de la derecha.

SQL

```
-- ¿El array [1, 2, 3, 4] CONTIENE a [2, 4]? -> TRUE  
SELECT ARRAY[1, 2, 3, 4] @> ARRAY[2, 4];
```

```
-- ¿El array [1, 2, 3] CONTIENE a [2, 5]? -> FALSE (falta el 5)  
SELECT ARRAY[1, 2, 3] @> ARRAY[2, 5];
```

B. Operador <@ (Está contenido en / Is Contained By)

Es el inverso de @>. Evalúa si el array de la izquierda **está totalmente incluido** dentro del array de la derecha.

SQL

```
-- ¿El array [2, 4] ESTÁ CONTENIDO en [1, 2, 3, 4]? -> TRUE  
SELECT ARRAY[2, 4] <@ ARRAY[1, 2, 3, 4];
```

C. Operador && (Solapamiento / Overlap)

Evalúa si dos arrays tienen **al menos un elemento en común** (intersección no vacía).

SQL

```
-- ¿Tienen algún elemento en común? -> TRUE (el 3 coincide)  
SELECT ARRAY[1, 2, 3] && ARRAY[3, 4, 5];
```

```
-- ¿Tienen algún elemento en común? -> FALSE  
SELECT ARRAY[1, 2] && ARRAY[3, 4];
```


3. Ejemplo Práctico: Etiquetado y Búsqueda de Productos

Supongamos que añadimos una columna de etiquetas (tags TEXT[]) a la tabla ventas.productos para categorizarlos:

SQL

-- 1. Agregar la columna de tipo Array

```
ALTER TABLE ventas.productos ADD COLUMN IF NOT EXISTS etiquetas TEXT[];
```

-- 2. Actualizar algunos registros con etiquetas

```
UPDATE ventas.productos SET etiquetas = ARRAY['electronica', 'computo', 'premium'] WHERE id = 1;
```

```
UPDATE ventas.productos SET etiquetas = ARRAY['perifericos', 'computo', 'oficina'] WHERE id = 2;
```

Consultas Usando los Operadores:

1. Buscar productos que tengan TODAS estas etiquetas (Uso de @>)

Queremos productos que sean estrictamente 'computo' Y 'premium':

SQL

```
SELECT id, nombre, etiquetas
```

```
FROM ventas.productos
```

```
WHERE etiquetas @> ARRAY['computo', 'premium'];
```

2. Buscar productos que tengan AL MENOS UNA de estas etiquetas (Uso de &&)

Queremos productos que sean 'oficina' O 'premium':

SQL

```
SELECT id, nombre, etiquetas
```

```
FROM ventas.productos
```

```
WHERE etiquetas && ARRAY['oficina', 'premium'];
```

4. Resumen Visual de Operadores

Operador	Nombre	Ejemplo	Descripción
=	Igualdad	ARRAY[1,2] = ARRAY[1,2]	Devuelve TRUE si son exactamente iguales en orden y contenido.
@>	Contiene	A @> B	Devuelve TRUE si A incluye todos los elementos de B.
<@	Está contenido	A <@ B	Devuelve TRUE si todos los elementos de A existen en B.

Operador	Nombre	Ejemplo	Descripción
&&	Solapamiento	A && B	Devuelve TRUE si A y B comparten al menos un elemento.
`		`	Concatenación

Tip de Rendimiento: En PostgreSQL puedes crear **índices GIN** sobre columnas de tipo Array (CREATE INDEX idx_productos_etiquetas ON ventas.productos USING GIN (etiquetas);). Esto hace que las consultas con los operadores @>, <@ y && sean extremadamente rápidas incluso en tablas con millones de filas.

funciones y procedimientos que marcan la diferencia en seguridad, rendimiento y diseño de software:

1. Modificadores de Seguridad: SECURITY DEFINER vs SECURITY INVOKER

Por defecto, una función o procedimiento se ejecuta con los **permisos del usuario que lo llama** (SECURITY INVOKER). Si un usuario de la aplicación no tiene permiso SELECT directo sobre una tabla interna, la función fallará.

Si declaras la función como **SECURITY DEFINER**, esta se ejecutará con los **permisos del usuario que la creó** (por ejemplo, postgres).

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_cambiar_password(p_user_id INT, p_pass TEXT)
RETURNS VOID
SECURITY DEFINER -- Se ejecuta con privilegios del creador
SET search_path = ventas, pg_temp -- ¡OBLIGATORIO por seguridad!
AS $$
BEGIN
    UPDATE ventas.usuarios SET password_hash = md5(p_pass) WHERE id = p_user_id;
END;
$$ LANGUAGE plpgsql;
```

Regla de Seguridad Crítica: Al usar SECURITY DEFINER, **siempre** debes definir de forma explícita el SET search_path dentro de la función para evitar ataques de inyección por sustitución de esquemas (*Search Path Hijacking*).

2. Volatilidad de Funciones: VOLATILE, STABLE e IMMUTABLE

PostgreSQL utiliza modificadores de volatilidad para que el optimizador de consultas decida si puede **guardar en caché** el resultado de una función o si debe reevaluarla por cada fila:

- **VOLATILE (Por defecto):** La función puede modificar la base de datos o devolver resultados distintos incluso con los mismos parámetros (ej. usa random(), now(), SELECTs).
- **STABLE:** No modifica la base de datos y garantiza devolver el mismo resultado **dentro de la misma transacción/consulta** para los mismos argumentos.
- **IMMUTABLE:** Promete que jamás cambiará la base de datos y que **para los mismos argumentos SIEMPRE devolverá el mismísimo resultado** (ej. abs(), operaciones matemáticas pura). **Solo las funciones IMMUTABLE se pueden usar para crear índices expresionales.**

SQL

```
-- Ejemplo IMMUTABLE: cálculo matemático puro
CREATE OR REPLACE FUNCTION ventas.fn_iva_producto(p_precio NUMERIC)
RETURNS NUMERIC
IMMUTABLE -- Permite a Postgres optimizar la ejecución en masa o usarla en índices
AS $$
BEGIN
    RETURN p_precio * 0.21;
END;
$$ LANGUAGE plpgsql;
```

3. Retornar Múltiples Valores sin Tablas (OUT Parameters)

Además de usar RETURNS TABLE(...), puedes declarar parámetros con el modificador **OUT** o **INOUT** para retornar múltiples valores directamente desde una función:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_estadisticas_cliente(
    IN p_cliente_id INT,
    OUT total_compras INT,
    OUT monto_total NUMERIC
) AS $$
BEGIN
    SELECT COUNT(*), COALESCE(SUM(p.cantidad * prod.precio), 0)
    INTO total_compras, monto_total
    FROM ventas.pedidos p
    JOIN ventas.productos prod ON p.producto_id = prod.id
```

```
WHERE p.cliente_id = p_cliente_id;
END;
$$ LANGUAGE plpgsql STABLE;
```

Invocación:

SQL

```
SELECT * FROM ventas.fn_estadisticas_cliente(1);
-- Devuelve una sola fila con dos columnas: total_compras | monto_total
```

4. SQL Dinámico con EXECUTE (Uso con Cuidado)

A veces necesitas construir consultas SQL cuyos nombres de tabla, columna o criterios de ordenamiento no se conocen hasta el momento de la ejecución. En PL/pgSQL, la orden EXECUTE permite ejecutar cadenas de texto como SQL dinámico.

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_contar_filas_tabla(p_nombre_tabla TEXT)
RETURNS BIGINT AS $$
DECLARE
    v_total BIGINT;
BEGIN
    -- Se usa quote_ident() para prevenir SQL Injection en nombres de objetos
    EXECUTE 'SELECT COUNT(*) FROM ventas.' || quote_ident(p_nombre_tabla)
    INTO v_total;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql STABLE;
```

Buenas Prácticas:

- Usa **quote_ident()** para nombres de esquemas, tablas o columnas.
- Usa la cláusula **USING** (EXECUTE 'SELECT ... WHERE id = \$1' USING p_id;) para pasar parámetros literales de forma segura.

5. Sobrecarga de Funciones (*Function Overloading*)

En PostgreSQL puedes definir varias funciones con el **mismo nombre**, siempre y cuando la firma de tipos de sus parámetros (IN) sea diferente:

SQL

```
-- Versión 1: Buscar cliente por ID (Entero)
CREATE OR REPLACE FUNCTION ventas.fn_buscar_cliente(p_id INT)
RETURNS TABLE(id INT, nombre VARCHAR) AS $$
BEGIN
    RETURN QUERY SELECT c.id, c.nombre FROM ventas.clientes c WHERE c.id = p_id;
```

```

END;
$$ LANGUAGE plpgsql STABLE;

-- Versión 2: Buscar cliente por Email (Texto)
CREATE OR REPLACE FUNCTION ventas.fn_buscar_cliente(p_email TEXT)
RETURNS TABLE(id INT, nombre VARCHAR) AS $$
BEGIN
    RETURN QUERY SELECT c.id, c.nombre FROM ventas.clientes c WHERE c.email = p_email;
END;
$$ LANGUAGE plpgsql STABLE;

```

6. Funciones Eventual/Disparadoras: *Triggers*

Aunque mencionamos excepciones en triggers anteriormente, no vimos la estructura de una **Función Trigger**: una función sin parámetros explícitos que devuelve el tipo especial TRIGGER y accede a variables automáticas como NEW, OLD, TG_OP (INSERT, UPDATE, DELETE).

SQL

```

CREATE OR REPLACE FUNCTION ventas.fn_trg_auditoria_productos()
RETURNS TRIGGER AS $$
BEGIN
    IF (TG_OP = 'UPDATE') THEN
        RAISE NOTICE 'El producto % cambió su precio de % a %',
            OLD.id, OLD.precio, NEW.precio;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_auditar_precio
AFTER UPDATE ON ventas.productos
FOR EACH ROW
EXECUTE FUNCTION ventas.fn_trg_auditoria_productos();

```

Resumen de la "Matriz de Selección"

Requisito	Herramienta Recomendada
¿Necesitas ejecutar COMMIT o ROLLBACK manual durante el proceso?	CREATE PROCEDURE
¿Calculas un valor o tabla para usarlo directo dentro de un SELECT?	CREATE FUNCTION
¿La función se ejecuta con permisos de administrador pero la llama un usuario básico?	SECURITY DEFINER

Requisito	Herramienta Recomendada
¿La consulta depende del valor de una columna dinámica enviada como string?	EXECUTE (SQL Dinámico)
¿Necesitas reaccionar automáticamente ante un INSERT, UPDATE o DELETE?	TRIGGER

funciones de agregación, las funciones de ventana (*window functions*) y el trabajo con datos no estructurados (JSON y JSONB)

En PostgreSQL 18, el dominio de las **funciones de agregación**, las **funciones de ventana (*window functions*)** y el trabajo con datos no estructurados (**JSON** y **JSONB**) es lo que separa a un usuario básico de un desarrollador o DBA avanzado.

A continuación veremos ambos bloques con explicaciones claras, sintaxis moderna y ejemplos prácticos basados en nuestro esquema ventas.

PARTE 1: Funciones de Agregación vs. Funciones de Ventana

1. Funciones de Agregación (GROUP BY)

Las funciones de agregación (SUM, AVG, COUNT, MAX, MIN) toman múltiples filas y las **colapsan o resumen en una sola fila por grupo**.

SQL

```
-- Resumen total de ventas por producto
SELECT
    p.nombre AS producto,
    COUNT(ped.id) AS total_pedidos,
    SUM(ped.cantidad) AS unidades_vendidas,
    SUM(ped.cantidad * p.precio) AS ingresos_totales
FROM ventas.pedidos ped
JOIN ventas.productos p ON ped.producto_id = p.id
GROUP BY p.nombre;
```

2. Funciones de Ventana (OVER (...))

A diferencia de GROUP BY, **las funciones de ventana NO colapsan las filas**. Calculan un valor sobre un conjunto de filas relacionadas (la "ventana"), pero **mantienen la identidad de cada fila individual intacta**.

Sintaxis general:

\$\$\text{FUNCION}() \text{ OVER } (\text{PARTITION BY } \text{columna_grupo} \text{ ORDER BY } \text{columna_orden})\$\$

Principales Funciones de Ventana:

- ROW_NUMBER(): Asigna un número secuencial único a cada fila dentro de la ventana.
- RANK() / DENSE_RANK(): Otorga una posición o ranking (útil para Tops/Podios).
- SUM(...) OVER(...): Acumulados corridos (*running totals*).
- LAG(columna) / LEAD(columna): Accede a la fila **anterior** o **siguiente** dentro de la ventana (ideal para comparar ventas contra el mes previo).

Ejemplo Práctico: Running Totals y Ranking de Pedidos

Analizaremos los pedidos de cada cliente calculando:

1. El número de pedido secuencial por cliente.
2. El **acumulado monetario corriendo** (*running total*) a medida que el cliente realiza más compras.
3. El total general del cliente replicado en cada fila sin perder el detalle.

SQL

SELECT

```
ped.id AS pedido_id,  
c.nombre AS cliente,  
c.region,  
ped.fecha,  
(ped.cantidad * prod.precio) AS total_pedido,
```

-- 1. Secuencia de pedidos por cliente

```
ROW_NUMBER() OVER (  
    PARTITION BY ped.cliente_id  
    ORDER BY ped.fecha  
) AS num_pedido_cliente,
```

-- 2. Acumulado corriendo (Running Total) de gasto por cliente

```
SUM(ped.cantidad * prod.precio) OVER (  
    PARTITION BY ped.cliente_id  
    ORDER BY ped.fecha  
) AS acumulado_historico_cliente,
```

-- 3. Total general gastado por la región completa (sin colapsar filas)

```
SUM(ped.cantidad * prod.precio) OVER (  
    PARTITION BY c.region  
) AS total_region
```

FROM ventas.pedidos ped

JOIN ventas.clientes c ON ped.cliente_id = c.id

JOIN ventas.productos prod ON ped.producto_id = prod.id;

PARTE 2: JSON y JSONB en PostgreSQL 18

PostgreSQL soporta dos tipos de datos nativos para almacenar datos estructurados en formato JSON:

Característica	JSON	JSONB (Binary)
Formato de almacenamiento	Texto plano tal cual se envía.	Formato binario descomprimido y optimizado.
Velocidad de inserción	Ligera y ligeramente más rápida.	Mínimamente más lenta por el procesamiento inicial.
Velocidad de consulta	Más lenta (debe parsear el texto cada vez).	Ultra rápida (acceso directo a llaves).
Indexación	No soporta índices avanzados.	Soporta índices GIN (búsquedas instantáneas).
Espacio / Duplicados	Conserva espacios y llaves duplicadas.	Elimina espacios, reordena llaves y quita duplicados.

Regla de oro: Usa siempre **JSONB** a menos que tengas un caso de uso muy específico que requiera conservar exactamente el texto plano original.

1. Operadores Esenciales de JSONB

Operador	Tipo de retorno	Descripción	Ejemplo
->	jsonb	Extrae un campo por clave como un objeto JSONB.	data->'cliente'
->>	text	Extrae un campo por clave como texto plano .	data->>'nombre'
#>	jsonb	Extrae en una ruta anidada (<i>path</i>).	data#>'{dir, ciudad}'

Operador	Tipo de retorno	Descripción	Ejemplo
#>>	text	Extrae en ruta anidada como texto plano .	data#>>'{dir, ciudad}'
@>	boolean	¿El JSONB de la izquierda contiene el de la derecha?	detalles @>'{"color": "Negro"}'

2. Ejemplo Práctico: Tabla con Metadatos Flexibles (JSONB)

Añadiremos especificaciones técnicas dinámicas a la tabla ventas.productos mediante un campo JSONB.

SQL

```
-- 1. Agregar columna JSONB
ALTER TABLE ventas.productos ADD COLUMN IF NOT EXISTS especificaciones JSONB;
```

```
-- 2. Insertar o actualizar especificaciones heterogéneas
```

```
UPDATE ventas.productos
SET especificaciones = '{
  "marca": "Dell",
  "ram_gb": 16,
  "pantalla": {"pulgadas": 15.6, "tipo": "OLED"},
  "puertos": ["USB-C", "HDMI", "Thunderbolt"]}
'
WHERE id = 1;
```

```
UPDATE ventas.productos
SET especificaciones = '{
  "marca": "Logitech",
  "inalambrico": true,
  "mecanico": true,
  "puertos": ["USB-C"]}
'
WHERE id = 2;
```

3. Consultas en Campos JSONB

A. Extraer campos simples y anidados

SQL

```
SELECT
  nombre,
  precio,
  especificaciones->>'marca' AS marca,
  especificaciones#>>'{pantalla, tipo}' AS tipo_pantalla,
  (especificaciones->>'ram_gb')::INT AS ram_gb -- Convertir a número para operaciones
FROM ventas.productos
```

WHERE especificaciones IS NOT NULL;

B. Búsqueda por Contención (@>)

Busca todos los productos que contengan la marca "Logitech" sin importar qué otras llaves tengan:

SQL

```
SELECT nombre, especificaciones
FROM ventas.productos
WHERE especificaciones @> '{"marca": "Logitech"}';
```

4. Funciones de Conversión JSON Nativas

PostgreSQL permite convertir filas y agregados de SQL a objetos JSON y viceversa:

A. Convertir Filas SQL a Objetos JSON (jsonb_build_object, to_jsonb)

SQL

```
-- Crear una estructura JSON personalizada desde la base de datos
SELECT jsonb_build_object(
    'id_pedido', p.id,
    'comprador', c.nombre,
    'region', c.region,
    'total', (p.cantidad * prod.precio)
) AS pedido_json
FROM ventas.pedidos p
JOIN ventas.clientes c ON p.cliente_id = c.id
JOIN ventas.productos prod ON p.producto_id = prod.id;
```

B. Agrupar Filas en un Array JSON (jsonb_agg)

Excelente para construir respuestas que alimentarán una API REST directamente desde la base de datos:

SQL

```
-- Generar un JSON con todos los pedidos agrupados por cliente
SELECT
    c.nombre AS cliente,
    jsonb_agg(
        jsonb_build_object(
            'producto', prod.nombre,
            'cantidad', p.cantidad,
            'fecha', p.fecha
        )
    ) AS historial_compras_json
FROM ventas.clientes c
JOIN ventas.pedidos p ON c.id = p.cliente_id
JOIN ventas.productos prod ON p.producto_id = prod.id
GROUP BY c.nombre;
```

5. Indexación de JSONB para Alto Rendimiento

Para buscar velozmente en millones de registros JSONB, creamos un **Índice GIN**:

SQL

```
-- Crea un índice sobre toda la estructura del JSONB  
CREATE INDEX idx_productos_especificaciones_gin  
ON ventas.productos USING GIN (especificaciones);
```

Con este índice activo, consultas como:

```
SELECT * FROM ventas.productos WHERE especificaciones @> '{"inalambrico":  
true}';
```

se ejecutarán en milisegundos sin importar el tamaño de la tabla.